

TDOM Engine かんたん全体像

まず読む短い版

目次

まず全体像	1
0.1 四つの道	1
0.2 何が新しいのか	1
0.3 編集したときに起きること	2
0.4 なぜ LaTeX 執筆体験が変わるのか	2
0.5 最初に覚える言葉	2

まず全体像

このエンジンは、一言でいうと、

編集直後は速い仮表示を出し、裏で本物の `LaTeX` 出力に追いついたら正しい表示へ差し替えるエンジンである。

普通の `LaTeX` 執筆では、少し直すたびに文書全体を再コンパイルし、PDF を待ち、該当ページを探し直す。このエンジンはそこを分解している。

0.1 四つの道

表示には四つの道がある。先に日本語名で掴めばよい。

日本語での役割	実装名	何をするか
速い仮表示	checkpoint	編集した近くのブロックだけを、常駐 <code>TeX</code> で素早く組む
正しい全体出力	canonical	普通の <code>lualatex</code> で文書全体をコンパイルし、正本 PDF を作る
危ない部分の画像差し替え	exact chunk	数式・特殊環境など、仮表示が危ない部分だけ本物の画像に置き換える
ページ単位の増分 正本	shipping chain	<code>TDOM_SHIP=1</code> のとき、実際のページ出力を途中から作り直す

普段の体験では、最初に「速い仮表示」が見える。その後、「正しい全体出力」や「危ない部分の画像差し替え」が追いついて、見た目が本物の `LaTeX` に寄っていく。

0.2 何が新しいのか

新しさは、`LaTeX` を別物に置き換えていない点にある。

このエンジンは、ブラウザ上で独自の組版をして「`LaTeX` っぽく」見せているだけではない。中では本物の `LuaLaTeX` を動かし、`TeX` の途中状態を保存し、必要な場所だけ再利用している。

そのため、執筆中の体験は速くなるが、最終的な正しさは普通の LaTeX コンパイルに戻せる。

0.3 編集したときに起きること

ユーザーが本文を少し直すと、エンジンは次の順で動く。

1. ソースを小さなブロックに分ける。
2. どのブロックが変わったかを見る。
3. 変わっていない手前の TeX 状態を再利用する。
4. 変わった近くを常駐 TeX に読ませる。
5. すぐ表示できる形にページを組み直す。
6. 裏で本物の LaTeX コンパイルを走らせる。
7. 仮表示が危ない部分は、本物の画像に差し替える。

つまり、毎回文書全体を待つのではなく、まず近くを直して見せる。

0.4 なぜ LaTeX 執筆体験が変わるのか

LaTeX 執筆でつらいのは、入力と思考の間に「コンパイル待ち」が入ることである。

このエンジンでは、編集直後の確認を文書全体の再コンパイルから切り離す。だから、長い文書でも「書く、見る、直す」の往復が軽くなる。

さらに、危ない構文を無理に近似しない。表、複雑な数式、特殊な環境などで仮表示が嘘をつきそうなときは、その部分を本物の LaTeX 出力に逃がす。速さだけでなく、間違った表示を見せ続けないことも重要な機能である。

0.5 最初に覚える言葉

最初は次の対応だけ覚えればよい。

言葉	意味
checkpoint	TeX の途中状態を保存したもの
block	編集・再組版の単位に分けた本文
canonical	普通の LaTeX コンパイルによる正しい出力
exact chunk	正確な画像として貼る部分
opaque	仮表示をあきらめ、正本表示へ寄せる状態
shipping chain	ページ単位で実際の LaTeX 出力を作り直す任意機能

このあと細部を読むなら、まず第 2 章で全体のファイル配置を見て、第 3 章で速い仮表示の仕組みを読む。